UNIVERSIDAD PERUANA UNIÃ“N
Facultad de IngenierÃa y Arquitectura Â· Escuela Profesional de IngenierÃa de Sistemas

Actividad AutÃ³noma â€” SesiÃ³n 11: Seguridad Frontend SPA

Matriz de Pruebas de la SPA Segura, Aviso Previo a la ExpiraciÃ³n y PÃ¡gina Â«Mi cuentaÂ» (/api/v1/auth/me)

Unidad: II. Frontend SPA empresarial seguro	**SesiÃ³n:** 11 (Reto 02 AutÃ³nomo)
Estudiante: SebastiÃ¡n Gamarra	**Docente:** Reyna Barreto Benjamin David
Puntaje: 20 puntos (EvaluaciÃ³n de sesiÃ³n)	**TecnologÃas:** Angular 22 Â· TypeScript Â· Spring Boot 4 (JWT)
Repositorio: PharmaFrontendv1	**Rama Git:** feature/seguridad-spa-pruebas-gamarra

Parte A. Matriz de Pruebas de la SPA Segura (24 Casos Medidos)

Se diseÃ±Ã³ y ejecutÃ³ una matriz formal de 24 casos de prueba (16 casos base, 4 casos propios de grupos distintos y 4 casos de extensiÃ³n B). Todos los tokens JWT se encuentran debidamente enmascarados (eyJhbGciOi...) para cumplir con el estÃ¡ndar de confidencialidad:

ID	Grupo	Usuario	Pasos	Resultado Esperado	Resultado Obtenido	Estado	Evidencia
L-01	Login	AnÃ³nimo	1. Abrir /login. 2. Pulsar Ingresar vacÃo. 3. Digitar solo user.	Mensajes de validaciÃ³n; ninguna peticiÃ³n HTTP en Red.	Formulario marca touched; muestra avisos por campo; 0 peticiones.	**PASA**	L-01_login.png
L-02	Login	Varios	1. Usuario noexiste. 2. Clave incorrecta. 3. Usuario inactivo.	Mismo mensaje en los 3 casos; clave se borra, usuario se conserva.	Backend responde 401; SPA muestra mensaje genÃ©rico unificado.	**PASA**	L-02_login.png
L-03	Login	vendedor	1. Credenciales vÃ¡lidas. 2. Doble clic rÃ¡pido en Ingresar.	Una sola peticiÃ³n POST /auth/login (botÃ³n se deshabilita).	Signal enviando=true deshabilita el botÃ³n; exactamente 1 POST en Red.	**PASA**	L-03_login.png
L-04	Login	admin	1. /login?returnUrl=https://sitio-falso.com 2. Iniciar sesiÃ³n.	Va a /inicio, nunca a un sitio externo (Open Redirect mitigado).	destino() descarta la URL externa y navega estrictamente a /inicio.	**PASA**	L-04_login.png
L-05	Login (P)	supervisor	1. Digitar contraseÃ±a. 2. Conmutar Â«Mostrar contraseÃ±aÂ».	Input cambia type='password' a 'text' y viceversa de forma reactiva.	verClave signal conmuta el atributo type; valor Ãntegro al enviar.	**PASA**	L-05_login.png
S-01	SesiÃ³n	vendedor	1. Iniciar sesiÃ³n. 2. Pulsar F5. 3. Cerrar y abrir navegador.	Tras F5 sigue la sesiÃ³n; tras cerrar navegador se pide login.	sessionStorage sobrevive a recarga pero se elimina al cerrar navegador.	**PASA**	S-01_session.png
S-02	SesiÃ³n	almacen	1. En sessionStorage cambiar expiraEn a timestamp pasado. 2. F5.	La sesiÃ³n guardada se descarta y aparece el login.	leerSesionGuardada() detecta expiraciÃ³n, borra clave y retorna null.	**PASA**	S-02_session.png
S-03	SesiÃ³n	admin	1. Escribir texto no-JSON en pharmasoft.sesion. 2. F5.	La SPA no se rompe: muestra el login (try/catch seguro).	Bloque try/catch captura SyntaxError, limpia clave y redirige.	**PASA**	S-03_session.png
S-04	SesiÃ³n	supervisor	1. Abrir SPA en 2 pestaÃ±as. 2. Cerrar sesiÃ³n en una. 3. Navegar en otra.	Registrar comportamiento: sessionStorage es por pestaÃ±a.	PestaÃ±a 2 retiene sesiÃ³n local hasta que la API responde 401.	**PASA**	S-04_session.png

S-05	SesiÃ³n (P)	vendedor	1. Alterar sessionStorage aÃ±adiendo CATALOGO_ESCRIBIR. 2. F5 y crear.	SPA muestra botÃ³n, pero POST al backend responde 403 Forbidden.	Confirma que la seguridad real reside en la firma del servidor.	**PASA**	S-05_session.png
I-01	Interceptor	vendedor	1. GET /productos. 2. POST /auth/login. 3. PeticiÃ³n externa.	Solo las peticiones a la API llevan Authorization: Bearer.	authInterceptor filtra req.url.startsWith(apiUrl); login sin cabecera.	**PASA**	I-01_red.png
I-02	Interceptor	admin	1. Con sesiÃ³n, detener backend. 2. Ir a Productos.	Mensaje de error de conexiÃ³n; la sesiÃ³n NO se cierra (no es 401).	errorInterceptor preserva sesiÃ³n ante status 0; vista muestra aviso.	**PASA**	I-02_red.png
I-03	Interceptor	vendedor	1. Alterar 1 carÃ¡cter del JWT en sessionStorage. 2. Abrir Ventas.	401; login con Â«Su sesiÃ³n expirÃ³Â» y returnUrl=%2Fventas.	Firma JWT invÃ¡lida; errorInterceptor expulsa limpiamente al login.	**PASA**	I-03_red.png
I-04	Interceptor (P)	AnÃ³nimo	1. PeticiÃ³n directa a endpoint pÃºblico /api/health.	PeticiÃ³n sale limpia sin encabezado Authorization.	authInterceptor comprueba !token y no aÃ±ade cabecera espuria.	**PASA**	I-04_red.png
G-01	Guards	AnÃ³nimo	1. Sin sesiÃ³n escribir /productos/1/editar, /ventas/nueva, /inicio.	Login con el returnUrl de cada ruta protegida.	authGuard intercepta y redirige con returnUrl en queryParams.	**PASA**	G-01_guard.png
G-02	Guards	vendedor / almacen	1. Vendedor abre /categorias/nuevo. 2. Almacenero abre /ventas/nueva.	Â«403 Â· Acceso denegadoÂ» en las rutas sin permiso concedido.	permisoGuard intercepta data['permiso'] y envÃa a /acceso-denegado.	**PASA**	G-02_guard.png
G-03	Guards (P)	admin	1. Con sesiÃ³n activa, escribir /login en el navegador.	invitadoGuard impide abrir login y redirige a /inicio.	invitadoGuard evalÃºa autenticado()=true y devuelve UrlTree a /inicio.	**PASA**	G-03_guard.png
M-01	MenÃº/Acc	Los 4	1. Iniciar con admin, vendedor, almacen y supervisor.	Cada usuario ve exactamente lo que indica su tabla de permisos.	Sidebar y directivas *appSiPermiso adaptan vistas a cada perfil.	**PASA**	M-01_menu.png
M-02	MenÃº/Acc	admin $\rightarrow$ almacen	1. Salir como admin e ingresar como almacen sin recargar.	El menÃº cambia instantÃ¡neamente sin F5 (computed reactivo).	Signal sesionActual() se actualiza y recalcula computed() en vivo.	**PASA**	M-02_menu.png
E-01	401 / 403	vendedor	1. Quitar *appSiPermiso del botÃ³n Â«Dar de bajaÂ» y pulsar.	El backend responde 403; SPA muestra mensaje y sesiÃ³n continÃºa.	Backend rechaza DELETE con 403; errorInterceptor mantiene sesiÃ³n activa.	**PASA**	E-01_403.png
B-01	Aviso Exp	vendedor	1. Backend con expiracion=3min. 2. Esperar a faltar $\le$ 120s.	Aparece banner flotante Â«âš Su sesiÃ³n vence en mm:ssÂ».	computed porVencer activa el componente app-aviso-sesion en main-layout.	**PASA**	B-01_aviso.png
B-02	Aviso Exp	almacen	1. Observar el banner durante varios segundos continuos.	Cuenta regresiva disminuye segundo a segundo en formato mm:ss.	setInterval en AuthService actualiza signal segundosRestantes() en vivo.	**PASA**	B-02_aviso.png
B-03	Aviso Exp	supervisor	1. Clic en botÃ³n [Ã—]. 2. Probar Â«Cerrar sesiÃ³n ahoraÂ».	Â«Ã—Â» oculta aviso durante sesiÃ³n; botÃ³n cerrar ejecuta logout inmediato.	avisoOculto.set(true) oculta banner; cerrarSesion() destruye sesiÃ³n.	**PASA**	B-03_aviso.png
B-04	Aviso Exp	admin	1. Dejar correr el tiempo hasta llegar a 00:00.	Al llegar a cero, SPA expulsa a /login?expirada=1 con aviso.	setTimeout dispara cerrarSesion('expirada', url); aviso en card de login.	**PASA**	B-04_aviso.png

Parte B. Funcionalidades Desarrolladas (Reto 02)

B1. Aviso Previo a la ExpiraciÃ³n: Se implementÃ³ en AuthService el signal segundosRestantes y el computed porVencer (activo cuando quedan ≤ 120 s). El componente standalone shared/components/aviso-sesion muestra la cuenta regresiva en formato mm:ss, permitiendo cerrar sesiÃ³n inmediatamente o posponer el aviso visual mediante Â«Ã—Â» (usando avisoOculto). El temporizador se desaloja rigurosamente con clearInterval al cerrar sesiÃ³n para evitar fugas de memoria.

B2. PÃ¡gina Â«Mi cuentaÂ» (/api/v1/auth/me): Se habilitÃ³ la ruta protegida /mi-cuenta dentro del layout. Consume el endpoint GET /api/v1/auth/me a travÃ©s de obtenerPerfil(). La interfaz renderiza el avatar, nombre, usuario, roles asignados y una matriz de permisos

clasificada por mÃ³dulos (CatÃ¡logo, Clientes, Ventas, Reportes y Seguridad) con marcas âœ" y âœ˜. Incorpora detecciÃ³n en tiempo real de discrepancias: si el administrador alterÃ³ los permisos del usuario en la base de datos, permisosCambiaron despliega un banner de advertencia recomendando reiniciar sesiÃ³n.

Parte C. Informe de Seguridad del Frontend

C1. Resumen de EjecuciÃ³n por Grupos:

Grupo Evaluado	Casos Totales	Casos Pasan	Casos Fallan	Tasa de Ã‰xito
Login (L)	5	5	0	100%
SesiÃ³n (S)	5	5	0	100%
Interceptor (I)	4	4	0	100%
Guards (G)	3	3	0	100%
MenÃº y Acciones (M)	2	2	0	100%
Manejo 401 / 403 (E)	1	1	0	100%
Aviso ExpiraciÃ³n (B)	4	4	0	100%
TOTAL CONSOLIDADO	**24**	**24**	**0**	**100%**

C2. Fichas de Hallazgos y Vulnerabilidades Arquitecturales:

Hallazgo 01: ExposiciÃ³n de Token JWT en sessionStorage frente a Ataques XSS	**Severidad: Alta**
Evidencia: sessionStorage.getItem('pharmasoft.sesion')	**Causa Probable:** AuthService.ts (iniciar / leerSesionGuardada)
DescripciÃ³n: El token JWT se almacena en sessionStorage para persistir ante recargas F5. No obstante, al ser accesible mediante la API JavaScript del navegador, cualquier inyecciÃ³n de cÃ³digo XSS permite extraer el JWT de forma trivial y suplantar al usuario sin restricciÃ³n.	**CorrecciÃ³n Propuesta:** Migrar el almacenamiento del token a cookies HttpOnly, Secure y SameSite=Lax/Strict administradas por el backend con mecanismos de validaciÃ³n anti-CSRF.

Hallazgo 02: Falsa SensaciÃ³n de Seguridad por ManipulaciÃ³n de Permisos en Cliente (Frontend Bypass)	**Severidad: Media**
Evidencia: S-05_sesion_permisos_tamper.png	**Causa Probable:** AuthService.ts (tienePermiso) / Sidebar / Directivas
DescripciÃ³n: tienePermiso() evalÃºa el arreglo JSON guardado en sessionStorage. Un atacante puede alterar localmente sus permisos aÃ±adiendo CATALOGO_ESCRIBIR o USUARIOS_ADMINISTRAR y la SPA desplegarÃ¡ botones y formularios protegidos. Aunque el backend responde 403 al enviar peticiones reales, la UI expone la superficie interna.	**CorrecciÃ³n Propuesta:** Reconocer que el frontend jamÃ¡s debe considerarse un lÃmite de seguridad perimetral. Validar criptogrÃficamente los claims en servidor en cada peticiÃ³n.

Hallazgo 03: Ausencia de Mecanismo de RevocaciÃ³n Inmediata de Tokens en Servidor (Stateless JWT)	**Severidad: Media**
Evidencia: L-02_login.png / Casos de inhabilitaciÃ³n	**Causa Probable:** SecurityConfig.java / JwtAuthenticationFilter.java
DescripciÃ³n: Al operar con sesiones sin estado (STATELESS), si un administrador inhabilita a un empleado (usuario inactivo) o altera sus roles en la base de datos, el token emitido continuarÃ¡ siendo vÃ¡lido hasta que expire su tiempo de vida (60 minutos).	**CorrecciÃ³n Propuesta:** Incorporar una lista negra (blacklist) de tokens revocados en memoria compartida (Redis) o verificar la fecha de Ãºltima actualizaciÃ³n de credenciales del usuario.

C3. Preguntas de AnÃ¡lisis TÃ©cnico:

1. Â¿Por quÃ© el backend debe validar cada permiso si la SPA ya los oculta? (Evidencia E-01)

El cÃ³digo que se ejecuta en el navegador del cliente estÃ¡ completamente fuera del control de la organizaciÃ³n; cualquier usuario puede abrir las herramientas de desarrollador (F12), alterar el cÃ³digo JavaScript en memoria, remover directivas como *appSiPermiso o forzar peticiones fetch/XHR directas contra la API. El caso E-01 demuestra que cuando un usuario con rol VENDEDOR evade la interfaz y ejecuta la baja de un producto, es el backend quien rechaza la operaciÃ³n con cÃ³digo HTTP 403 Forbidden gracias a las reglas declarativas en SecurityConfig y anotaciones @PreAuthorize. La protecciÃ³n en el frontend responde Ãºnicamente a la experiencia de usuario (evitar frustraciones de navegaciÃ³n), mientras que la seguridad verÃdica, irrenunciable e inviolable reside exclusivamente en el servidor.

2. Â¿Ventajas y riesgos de sessionStorage frente a localStorage y HttpOnly? ElecciÃ³n para la botica.

sessionStorage aÃsla la sesiÃ³n por pestaÃ±a y la destruye automÃ¡ticamente al cerrar el navegador, evitando que en terminales compartidas de la botica el siguiente trabajador acceda a la sesiÃ³n anterior; sin embargo, sigue siendo vulnerable a robo de tokens vÃa XSS. localStorage retiene el token indefinidamente sin importar el reinicio del equipo, lo cual es inaceptable para computadoras multiusuario. La alternativa Ã³ptima y mÃ¡s robusta es una cookie con los atributos HttpOnly (inaccesible a scripts JS, anulando la exfiltraciÃ³n por XSS), Secure (transmisiÃ³n forzada por HTTPS) y SameSite=Strict/Lax. Para la botica elegirÃa cookies HttpOnly emitidas por Spring Boot con protecciÃ³n anti-CSRF, garantizando confidencialidad absoluta incluso si existiera una vulnerabilidad XSS en el frontend.

3. Â¿Por quÃ© errorInterceptor cierra la sesiÃ³n ante 401 pero no ante 403? Â¿QuÃ© pasarÃa al revÃ©s?

El cÃ³digo HTTP 401 Unauthorized indica que la autenticaciÃ³n ha fracasado: no existe token, la firma es corrupta o la vigencia expirÃ³; por ende, es imprescindible destruir los datos caducos y redirigir al login. Por el contrario, el cÃ³digo HTTP 403 Forbidden certifica que el usuario SÃ■ estÃ¡ correctamente autenticado e identificado, pero carece de autorizaciÃ³n para un recurso puntual; cerrar la sesiÃ³n ante un 403 provocarÃa una experiencia desastrosa expulsando arbitrariamente a trabajadores legÃtimos de la aplicaciÃ³n por el simple hecho de hacer clic en una funciÃ³n fuera de su competencia. Si se hiciera al revÃ©s, usuarios con tokens vencidos quedarÃan en bucles infinitos de error sin poder reingresar y usuarios autenticados perderÃan su sesiÃ³n de trabajo constantemente.

4. Â¿QuÃ© es una redirecciÃ³n abierta (Open Redirect) y cÃ³mo la evita returnUrl? (Caso L-04)

Una redirecciÃ³n abierta es una vulnerabilidad en la cual un atacante manipula un parÃ¡metro de retorno (como returnUrl=https://sitio-malicioso.com) para que, luego de que la vÃctima se autentica en el dominio legÃtimo, la aplicaciÃ³n la transfiera automÃ¡ticamente a un portal de phishing idÃ©ntico diseÃ±ado para recolectar informaciÃ³n o distribuir malware. En PharmaSoft se mitiga en el mÃ©todo destino() mediante la validaciÃ³n: url && url.startsWith('/') && !url.startsWith('//'). Esto garantiza que la SPA Ãºnicamente navegue a rutas internas que comiencen con una barra simple (/ventas, /inicio) y rechace cualquier esquema de protocolo relativo (//sitio.com) o URLs absolutas (https://), neutralizando cualquier intento de redirecciÃ³n externa maliciosa.

Parte D. Versionamiento y Lista de Cotejo de Entrega

Commits en la rama feature/seguridad-spa-pruebas-gamarra:
â€¢ ef70622: test(auth): matriz de pruebas de la SPA segura
â€¢ 2596f63: feat(auth): aviso previo a la expiraciÃ³n de la sesiÃ³n
â€¢ 3f4dc9e: feat(auth): pÃ¡gina Mi cuenta con /auth/me
Pull Request hacia develop: https://github.com/Gamarra191207/PharmaFrontendv1/pull/new/feature/seguridad-spa-pruebas-gamarra

N.Âº	Criterio Observable	Pts	Estado
1	La matriz tiene al menos 20 casos, con los 16 base y 4 propios de grupos distintos.	2	SÃ■
2	Cada caso indica pasos, resultado esperado y obtenido, y su estado.	2	SÃ■
3	Casos de sesiÃ³n e interceptor (F5, sesiÃ³n vencida o daÃ±ada, token alterado, backend detenido) ejecutados con evidencia.	2	SÃ■
4	Casos de guards, menÃº y 403 cubren los cuatro usuarios activos y distinguen 401 de 403.	1	SÃ■
5	Cada caso tiene su evidencia nombrada con el ID y ninguna muestra un token completo.	1	SÃ■
6	El aviso aparece cuando quedan dos minutos o menos y muestra cuenta regresiva en mm:ss.	1.5	SÃ■
7	El aviso permite cerrar sesiÃ³n u ocultarse, y el temporizador se limpia al cerrar la sesiÃ³n.	1.5	SÃ■
8	PÃ¡gina Â«Mi cuentaÂ» protegida, consume /auth/me con estados de carga/error y muestra roles/permisos por mÃ³dulo.	2	SÃ■
9	Â«Mi cuentaÂ» detecta cuando los permisos del servidor difieren de los de la sesiÃ³n.	1	SÃ■
10	Las fichas de hallazgos indican causa probable y correcciÃ³n propuesta.	1.5	SÃ■
11	Las cuatro preguntas de anÃ¡lisis se responden con argumentos y referencias al cÃ³digo.	1.5	SÃ■
12	La rama tiene al menos tres commits en dÃas distintos, documentos en docs/ y solicitud de incorporaciÃ³n.	2	SÃ■

TOTAL	**Puntaje de evaluaciÃ³n consolidado**	**20**	**20/20**